

ResolveAI

AI-Powered Help Desk and Support Platform

Software Requirements Specification (SRS) and Complete Project Proposal

Finalized Technology Stack

Django web portal with AdminLTE2 templates, FastAPI for AI/ML services, MySQL database, Docker Compose for development, and GitLab/Jira for engineering workflow.

Internship Duration: 8 Weeks | Team Size: 4 Fresh Graduates

Document Version: 1.0 | Prepared for Internship Batch 2026

Document Control

Item	Details
Project Name	ResolveAI - AI-Powered Help Desk and Support Platform
Document Type	Software Requirements Specification and Implementation Proposal
Primary Audience	Interns, mentor, reviewers, and project stakeholders
Project Duration	8 weeks
Team Size	4 interns working as one delivery team
Delivery Model	Incremental development with weekly demonstrations
Technology Stack	Django + AdminLTE2, FastAPI, MySQL, Docker Compose, GitLab, Jira
Version	1.0

How to Use This Document

- Treat this document as the baseline definition of what the team is expected to build.
- Use the functional requirements to create Jira epics, stories, tasks, and acceptance criteria.
- Use the RBAC matrix when implementing permissions in Django.
- Use the workflow and status-transition sections to implement the complete ticket lifecycle.
- Use the mock screens as low-fidelity references; the final UI may improve visually but should preserve the required actions.
- Any scope change should be discussed with the mentor and recorded in Jira before implementation.

Table of Contents

1. Executive Summary
2. Project Vision, Goals and Scope
3. Stakeholders and User Roles
4. Technology Architecture
5. Functional Modules
6. Role-Based Access Control
7. End-to-End Ticket Workflow
8. Detailed Functional Requirements
9. AI/ML Service Requirements
10. Database and Data Model
11. API Design
12. Admin Panel Requirements
13. Mock Screens
14. Base Project Boilerplate
15. Non-Functional Requirements
16. Security and Privacy
17. Testing Strategy
18. Deployment and DevOps
19. Project Delivery Plan
20. Acceptance Criteria and Definition of Done
21. Future Enhancements
22. Glossary

1. Executive Summary

ResolveAI is a web-based support ticket management platform enhanced with AI-assisted classification, prioritization, summarization, sentiment analysis, duplicate detection, and FAQ-based response suggestions. The system will allow end users to submit and track support requests, while support agents and administrators manage those requests through a controlled workflow.

The project is intentionally designed as a complete product rather than a collection of disconnected AI notebooks. The interns must deliver an integrated web portal, AI service layer, database, role-based permissions, administration interfaces, automated tests, Docker-based setup, documentation, and a final demonstration.

Core Principle

AI assists support staff; it does not make irreversible decisions automatically. Human users can review, correct, and override AI-generated categories, priorities, summaries, and suggested responses.

2. Project Vision, Goals and Scope

2.1 Vision

To create an intelligent, transparent, and easy-to-use help desk system that reduces manual triage effort and improves response consistency while keeping humans in control.

2.2 Business Goals

- Reduce the time required to categorize and prioritize incoming tickets.
- Give users visibility into the status and history of their requests.
- Help agents respond faster using relevant FAQ content and AI suggestions.
- Provide management with operational dashboards and workload insights.
- Create a realistic team project that covers full-stack development, AI integration, testing, security, and deployment.

2.3 MVP Scope

Included in MVP	Out of Scope for MVP
User registration, login, profile and role-based access	Native mobile applications
Ticket creation, conversation, assignment and lifecycle	Real-time voice calling or call-center integration
AdminLTE2-based user and admin portal	Automatic sending of AI replies without agent approval
AI category, priority, sentiment and summary	Advanced custom ML model training on large private datasets
FAQ knowledge base and suggested answers with sources	Complex multi-tenant billing and subscription management
Search, filters, dashboard and audit history	Enterprise SSO, LDAP, or Active Directory integration
Docker Compose local environment and deployment guide	High-availability production infrastructure

3. Stakeholders and User Roles

Role	Primary Responsibilities
Requester / End User	Create tickets, view own tickets, add replies, review status, confirm resolution, reopen within policy.

Role	Primary Responsibilities
Support Agent	View assigned or permitted tickets, communicate with users, use AI suggestions, update status, add internal notes.
Support Manager	Monitor queues, assign/reassign tickets, override priority/category, review SLA and performance dashboards.
System Administrator	Manage users, roles, categories, FAQ documents, system settings, audit logs and access permissions.
AI Service	Return recommendations and analysis; never act as a human user or bypass authorization.
Mentor / Product Owner	Approve scope, review weekly progress, accept completed work and prioritize backlog.

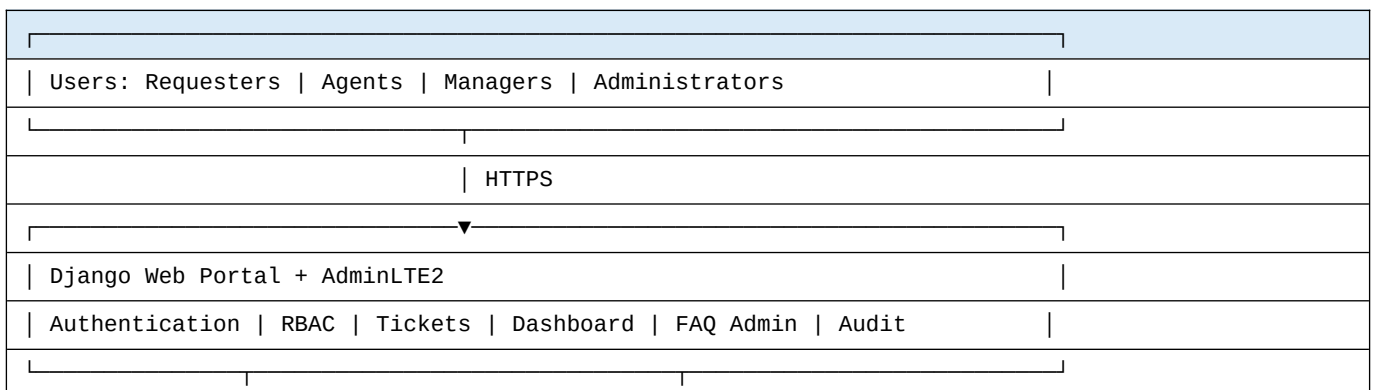
4. Technology Architecture

4.1 Final Technology Stack

Layer	Technology	Purpose
Web application	Django	Authentication, RBAC, ticket workflows, admin features, APIs, business logic.
UI template	AdminLTE2 converted into Django templates	Dashboard layout, navigation, forms, tables, cards and responsive interface.
AI/ML services	FastAPI	Classification, priority prediction, summarization, sentiment and FAQ retrieval endpoints.
Database	MySQL	Persistent storage for users, tickets, messages, audit history, FAQs and AI results.
Containerization	Docker and Docker Compose	Consistent local setup and service orchestration.
Source control	GitLab	Repository, branches, merge requests, reviews and CI/CD.
Work management	Jira	Epics, stories, tasks, bugs, sprint tracking and acceptance evidence.

4.2 Logical Architecture

System Context Diagram



ORM / SQL		REST/JSON	
MySQL Database		FastAPI AI Service	
Operational + audit data		Classify Priority Summary RAG	

4.3 Service Responsibilities

- Django owns all business workflow and authorization decisions.
- FastAPI receives text and returns analysis results; it does not directly change ticket status or permissions.
- MySQL is the system of record for users, tickets, messages, configuration and AI outputs.
- AdminLTE2 is converted into reusable Django base templates, partials and components.
- Docker Compose starts the web, AI and database services consistently for every team member.

5. Functional Modules

Module	Description
Authentication and User Management	Registration, login, logout, password reset, profile, account status, role assignment.
RBAC and Authorization	Role permissions, object-level access, protected views, menu visibility, audit of privileged actions.
Ticket Management	Create, view, update, assign, categorize, prioritize, reply, resolve, close and reopen tickets.
Conversation and Notes	Public replies, agent replies, internal notes, chronological timeline, attachments as optional extension.
AI Analysis	Category, priority, sentiment, summary, similar tickets and confidence metadata.
Knowledge Base / FAQ	Manage FAQ documents, sections, tags, active status, retrieval and source-backed suggestions.
Admin Panel	Users, roles, categories, teams, FAQs, ticket queues, AI results, audit logs and settings.
Dashboard and Reporting	Ticket counts, trends, workload, status distribution, category distribution, response metrics.
Notifications	In-app notifications for assignments, replies and status changes; email is optional.
Audit and Activity History	Record key actions with actor, timestamp, old value, new value and related ticket.
System Health	Web health endpoint, AI health endpoint, database connectivity and basic operational logging.

6. Role-Based Access Control (RBAC)

RBAC must be enforced in backend code. Hiding a menu item in the UI is not sufficient. Django views, service functions, forms and APIs must verify permissions before returning or changing data.

Action	Requester	Agent	Manager	Administrator
Register and maintain own profile	Yes	Own profile	Own profile	All users
Create ticket	Yes	Yes	Yes	Yes
View own tickets	Yes	Yes	Yes	Yes
View all tickets	No	Assigned/team scope	Yes	Yes
Reply publicly	Own ticket	Permitted tickets	Yes	Yes
Add internal note	No	Yes	Yes	Yes
Assign/reassign ticket	No	No or limited	Yes	Yes
Change category	No	Yes	Yes	Yes
Change priority	No	Recommend/request	Yes	Yes
Resolve ticket	Confirm/feedback	Yes	Yes	Yes
Close ticket	Limited confirmation	Yes	Yes	Yes
Reopen ticket	Own within policy	Yes	Yes	Yes
Manage FAQ	No	View only	Approve/edit	Full
Manage users and roles	No	No	Limited team management	Full
View audit log	Own activity only	Ticket-level	Operational	Full
Configure system settings	No	No	Limited	Full

6.1 Suggested Django Permission Model

- Use Django authentication and groups as the base permission system.
- Create groups: Requester, Support Agent, Support Manager, System Administrator.
- Define model permissions such as view_ticket, add_ticket, change_ticket, assign_ticket, add_internal_note and manage_faq.
- Apply object-level filtering so requesters only see their own tickets and agents see assigned/team tickets.
- Create reusable permission decorators, mixins or service-layer checks rather than repeating logic in every view.
- Record privileged operations such as role changes, ticket reassignment and priority override in the audit log.

7. End-to-End Ticket Workflow

7.1 Ticket Statuses

Status	Meaning	Typical Owner
Open	Ticket has been submitted and awaits triage or assignment.	System / Manager
In Progress	An agent is actively working on the request.	Agent
Waiting for User	Agent needs additional information from requester.	Requester
Waiting for Internal Team	Resolution depends on another internal person/team.	Agent / Manager

Status	Meaning	Typical Owner
Resolved	Agent believes the issue is solved; requester may confirm or reopen.	Agent
Closed	Work is complete and ticket is finalized.	Agent / Manager / System policy
Reopened	Requester or authorized staff indicates the issue persists.	Agent / Manager
Cancelled	Ticket was created by mistake or is no longer required.	Requester / Manager

7.2 Complete Workflow

23. 1. Requester signs in and creates a ticket with title, description and optional category.
24. 2. Django validates the form, creates a unique ticket number and stores the ticket as Open.
25. 3. Django sends the ticket text to FastAPI asynchronously or synchronously with timeout protection.
26. 4. FastAPI returns recommended category, priority, sentiment, summary and confidence values.
27. 5. Django stores the AI result separately from the human-approved values.
28. 6. If AI is unavailable, the ticket remains valid and enters manual triage.
29. 7. Manager reviews the new-ticket queue, confirms or overrides category and priority, then assigns an agent.
30. 8. Assigned agent receives an in-app notification and changes the ticket to In Progress.
31. 9. Agent reviews the conversation, AI summary, sentiment and suggested FAQ answer with cited source.
32. 10. Agent may edit and send a public reply, or add an internal note visible only to support staff.
33. 11. If more information is required, the agent sets Waiting for User.
34. 12. When the requester replies, the system records the message and returns the ticket to In Progress or the configured queue.
35. 13. Agent resolves the issue and enters a resolution note.
36. 14. Requester confirms resolution, or reopens the ticket within the permitted time window.
37. 15. Ticket is closed manually or automatically according to policy.
38. 16. Every significant transition is added to ticket history and audit logs.
39. 17. Dashboard metrics are updated from stored operational data.

7.3 Status Transition Rules

Current Status	Allowed Next Statuses	Who Can Change
Open	In Progress, Cancelled	Manager, assigned agent, administrator
In Progress	Waiting for User, Waiting for Internal Team, Resolved, Cancelled	Agent, manager, administrator
Waiting for User	In Progress, Resolved, Cancelled	Agent; requester reply may trigger In Progress
Waiting for Internal Team	In Progress, Resolved, Cancelled	Agent, manager
Resolved	Closed, Reopened	Requester, agent, manager, administrator
Closed	Reopened (policy-controlled)	Manager, administrator; requester within allowed period
Reopened	In Progress, Waiting for User, Resolved	Agent, manager, administrator

8. Detailed Functional Requirements

ID	Requirement
FR-001	The system shall allow a visitor to register as a requester, subject to configured policy.
FR-002	The system shall authenticate users and maintain secure sessions.
FR-003	The system shall restrict every page and action according to RBAC permissions.
FR-004	A requester shall create a ticket containing a title, detailed description and optional category.
FR-005	The system shall generate a unique human-readable ticket reference.
FR-006	A requester shall view only tickets they created unless additional permission is granted.
FR-007	Support staff shall search and filter tickets by status, category, priority, assignee, requester and date.
FR-008	Authorized users shall assign and reassign tickets while preserving assignment history.
FR-009	The system shall support public replies and staff-only internal notes.
FR-010	The system shall enforce valid status transitions.
FR-011	FastAPI shall provide category recommendations with confidence information.
FR-012	FastAPI shall provide priority recommendations without directly changing the final human-controlled value.
FR-013	FastAPI shall provide sentiment analysis and a concise ticket summary.
FR-014	The system shall retrieve relevant FAQ content and show the source of any suggested answer.
FR-015	An agent shall be able to edit a suggested answer before sending it.
FR-016	The system shall continue ticket creation even when the AI service is unavailable.
FR-017	The admin panel shall manage users, roles, categories, priorities, FAQs and system settings.
FR-018	The system shall record ticket activity, status history, assignment history and privileged changes.
FR-019	The dashboard shall show operational metrics based on actual database records.
FR-020	The system shall expose health-check endpoints for the web and AI services.

9. AI/ML Service Requirements

9.1 AI Endpoints

Endpoint	Input	Output
POST /api/v1/classify	Title and description	Recommended category, confidence, model version
POST /api/v1/priority	Ticket text and optional metadata	Recommended priority, confidence, reasons/signals
POST /api/v1/sentiment	Ticket text or latest message	Label, score and risk flag
POST /api/v1/summarize	Ticket conversation	Short factual summary
POST /api/v1/suggest-answer	Question/ticket text	Suggested answer, source documents and relevance scores
POST /api/v1/similar-tickets	Ticket text	Similar ticket identifiers or indexed references
GET /health	None	Service and model readiness status

9.2 AI Guardrails

- AI outputs must be labeled as recommendations.
- Low-confidence results should be clearly indicated and may require manual triage.
- Generated summaries must remain factual and avoid adding information not present in the ticket.
- Suggested answers should be grounded in the approved FAQ knowledge base and include sources.
- AI failures, timeouts and invalid responses must be handled without breaking the main workflow.
- Store model/service version with each analysis result for traceability.
- Do not send passwords, secrets, tokens or unnecessary personal information to AI components.

9.3 Baseline Implementation Options

Feature	Suitable Internship Baseline
Classification	Rule-based baseline or TF-IDF + logistic regression; later compare with embeddings/LLM approach.
Priority	Keyword/rule baseline combined with classifier; final value remains human-controlled.
Sentiment	Pretrained sentiment pipeline or lightweight classifier.
Summarization	Extractive summary or controlled LLM call if approved and available.
FAQ retrieval	Sentence embeddings and vector similarity over approved FAQ chunks.
Similar tickets	Embedding similarity over resolved ticket summaries or sample dataset.

10. Database and Data Model

10.1 Core Entities

Entity	Important Fields
User	id, email, password hash, first_name, last_name, is_active, created_at
Role / Group	id, name, permissions
SupportTeam	id, name, manager, active
Ticket	id, ticket_number, requester_id, title, description, status, category_id, priority, assignee_id, team_id, created_at, updated_at, resolved_at, closed_at
TicketMessage	id, ticket_id, author_id, body, visibility, created_at
TicketStatusHistory	id, ticket_id, old_status, new_status, changed_by, reason, created_at
TicketAssignmentHistory	id, ticket_id, previous_assignee, new_assignee, changed_by, created_at
Category	id, name, description, active
FAQDocument	id, title, content/source path, version, active, created_by
FAQChunk	id, document_id, heading, text, embedding_reference, active
AIAnalysis	id, ticket_id, analysis_type, input_hash, result_json, confidence, model_version, status, created_at
AuditLog	id, actor_id, action, object_type, object_id, old_values, new_values, ip_address, created_at
Notification	id, user_id, type, message, read_at, related_ticket_id

10.2 Relationship Summary

- A requester can create many tickets.
- A ticket has many messages, status-history entries, assignment-history entries and AI analyses.
- A ticket belongs to one current category and may have one current assignee and support team.
- An FAQ document contains multiple searchable chunks.
- Users receive many notifications and may appear as actors in audit records.

10.3 Data Integrity Rules

- Ticket numbers must be unique.
- Every public or internal message must have an author and ticket.
- Internal notes must never be returned to requesters.
- Closing a ticket should store closed_at; resolving should store resolved_at.
- AI output should not overwrite the approved category or priority without an authorized human action.
- Deleting users should not destroy historical ticket and audit records; use deactivation or protected relationships.

11. API Design

11.1 Django Application Endpoints

Method and Path	Purpose	Minimum Role
POST /api/tickets	Create ticket	Authenticated requester
GET /api/tickets	List permitted tickets	Authenticated
GET /api/tickets/{id}	View permitted ticket	Authenticated with object access
POST /api/tickets/{id}/messages	Add public reply	Requester/agent with access
POST /api/tickets/{id}/internal-notes	Add staff-only note	Agent
POST /api/tickets/{id}/assign	Assign or reassign	Manager
POST /api/tickets/{id}/transition	Change ticket status	Role and transition dependent
POST /api/tickets/{id}/run-ai	Request/retry analysis	Agent/manager/admin
GET /api/dashboard/summary	Operational dashboard data	Agent/manager/admin
CRUD /api/admin/faqs	Manage FAQ content	Manager/admin

11.2 Standard Response and Error Rules

- Use JSON for service-to-service APIs.
- Validate all required fields and return meaningful 4xx responses for invalid requests.
- Return 401 for unauthenticated requests and 403 for authenticated users lacking permission.
- Return 404 without leaking the existence of records the user is not permitted to view.
- Use a consistent error structure containing code, message and optional field errors.
- Apply timeouts and retry policy when Django calls FastAPI.

12. Admin Panel Requirements

The project should include both operational AdminLTE2 screens and selected Django Admin capabilities. The custom AdminLTE2 portal is the primary user-facing administration interface; Django Admin may support internal maintenance where appropriate.

Admin Area	Required Capabilities
Dashboard	Ticket counts, status cards, priority distribution, category chart, recent activity, workload by agent.
Ticket Queue	Filter, sort, search, bulk view, assignment, priority/category override, status transition.
User Management	Create/deactivate users, assign groups, reset access, view activity.
Role and Permission Management	View roles and permissions; changes restricted to administrators.
Categories and Priorities	Create, edit, order and activate/deactivate categories and configuration values.
FAQ Knowledge Base	Create/import documents, edit sections, activate versions, rebuild search index.
AI Review	View analyses, confidence, model version, failures and manual overrides.

Admin Area	Required Capabilities
Audit Log	Filter by actor, action, ticket, object and date; read-only.
System Settings	Reopen window, default status, AI timeout, notification settings and feature flags.

13. Mock Screens

These wireframes define the minimum content and actions. The team should implement them using AdminLTE2 components and reusable Django templates.

13.1 Login Screen

RESOLVEAI	[Help]
Sign in to continue	
Email	
Password	
☐ Remember me	[Forgot password?]
[Sign In]	
New requester? [Create account]	

13.2 Requester Dashboard

HEADER: ResolveAI My Tickets New Ticket Notifications Profile				
Open: 3	Waiting: 1	Resolved: 8		
[+ Create New Ticket]				
Filters: Status [All] Search [Apply]				

#RA-1042 Portal login issue In Progress High Updated 2h				
#RA-1038 Fee not reflected Waiting User Med Updated 1d				

13.3 New Ticket Screen

NEW SUPPORT TICKET	
Title *	
Category	[Select or leave for AI recommendation v]
Description *	[]
	[]
	[]

Attachment	[Choose file] (optional extension)
[Cancel]	[Submit Ticket]
Note: AI recommendations will be reviewed by support staff.	

13.4 Ticket Detail - Requester View

#RA-1042 - Portal login issue	Status: In Progress
Category: Account Access Priority: High Assigned: Support Team	

Timeline:	
[Requester] I cannot log in after resetting my password.	09:12
[Agent] Please confirm whether the reset email arrived.	09:40
[Requester] Yes, but the link says expired.	10:05

Add reply: [_____]	
[Send Reply]	[Confirm Resolution if resolved]

13.5 Agent Ticket Workspace

SIDEBAR: Dashboard My Queue All Tickets Knowledge Base
#RA-1042 - Portal login issue [In Progress v]
Requester: User Name Assignee: Agent A Team: Technical
Human Category: Account Access Human Priority: High
AI: Category Account Access (0.91) Priority High (0.84)
AI Sentiment: Negative Summary: Password reset link expired.

Conversation timeline...

Suggested answer [Source: FAQ / Password Reset / v2]
[Editable response text.....]
[Add Internal Note] [Send Reply] [Resolve Ticket]

13.6 Manager Ticket Queue

TICKET QUEUE
Filters: Status [Open] Category [All] Priority [All] Assignee [Any]
Search [_____] [Apply] [Reset]

[] #RA-1048 Payment issue Critical Unassigned 10 min ago
[] #RA-1047 Login problem High Agent B 25 min ago

[] #RA-1046 Transcript Medium Unassigned 1 hour ago
--

Bulk Action: [Assign to... v] [Apply]

13.7 Administration Dashboard

ADMIN DASHBOARD			
Open 42	Critical 4	Avg Res 6h	AI Failures 2
[Tickets by Status Chart]		[Tickets by Category Chart]	
[Agent Workload Table]		[Recent Audit Activity]	
Quick Links: Users Roles Categories FAQs AI Review Logs			

14. Base Project Boilerplate

14.1 Recommended Repository Structure

```

resolveai/
├── README.md
├── .env.example
├── .gitignore
├── docker-compose.yml
├── docs/
│   ├── architecture/
│   ├── api/
│   └── decisions/
├── web/                                # Django application
│   ├── Dockerfile
│   ├── manage.py
│   ├── requirements.txt
│   ├── config/
│   │   ├── settings/
│   │   │   ├── base.py
│   │   │   ├── local.py
│   │   └── production.py
│   │   ├── urls.py
│   │   └── wsgi.py
│   ├── apps/
│   │   ├── accounts/
│   │   ├── tickets/
│   │   ├── knowledge_base/
│   │   ├── ai_integration/
│   │   ├── notifications/
│   │   ├── audit/
│   │   └── dashboard/
│   └── templates/

```

```

|   |   |─ base.html
|   |   |─ partials/
|   |   |   |─ header.html
|   |   |   |─ sidebar.html
|   |   |   |─ footer.html
|   |   |   └─ messages.html
|   |   |─ accounts/
|   |   |─ tickets/
|   |   |─ dashboard/
|   |   └─ admin_portal/
|   |─ static/
|   |   |─ adminlte2/
|   |   |─ css/
|   |   |─ js/
|   |   └─ images/
|   └─ tests/
└─ ai_service/                # FastAPI application
    |─ Dockerfile
    |─ requirements.txt
    |─ app/
    |   |─ main.py
    |   |─ api/routers/
    |   |─ schemas/
    |   |─ services/
    |   |─ models/
    |   |─ retrieval/
    |   |─ core/config.py
    |   └─ tests/
    └─ data/
└─ scripts/
    |─ wait_for_db.sh
    └─ seed_demo_data.sh

```

14.2 Django Template Strategy for AdminLTE2

- Create base.html containing the common AdminLTE2 HTML shell.
- Extract the header, sidebar, footer, notifications and scripts into reusable partials.
- Use Django template blocks such as title, page_header, breadcrumbs, content and extra_scripts.
- Generate sidebar menu entries according to the current user permissions.
- Use reusable includes for status badges, priority badges, pagination and form errors.
- Keep AdminLTE2 vendor files separate from project-specific CSS and JavaScript.

14.3 Example Docker Compose Services

```

services:
  web:
    build: ./web
    command: python manage.py runserver 0.0.0.0:8000
    env_file: .env
    ports: ["8000:8000"]
    depends_on: [db, ai_service]

```

```

ai_service:
  build: ./ai_service
  command: uvicorn app.main:app --host 0.0.0.0 --port 8001 --reload
  env_file: .env
  ports: ["8001:8001"]

db:
  image: mysql:8.4
  environment:
    MYSQL_DATABASE: resolveai
    MYSQL_USER: resolveai
    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  ports: ["3306:3306"]
  volumes: ["mysql_data:/var/lib/mysql"]

volumes:
  mysql_data:

```

14.4 Initial Jira Epics

Epic	Example Stories
EPIC 1 - Foundation	Repository, Docker, Django setup, FastAPI setup, MySQL, AdminLTE2 base template.
EPIC 2 - Accounts and RBAC	Login, profile, groups, permissions, object-level access, menu authorization.
EPIC 3 - Ticket Workflow	Create ticket, queue, detail, assignment, replies, notes, transitions, history.
EPIC 4 - AI Services	Classification, priority, sentiment, summary, FAQ retrieval, health checks.
EPIC 5 - AI Integration	Django client, timeout/fallback, storage, retries, agent review interface.
EPIC 6 - Admin and Reporting	Users, categories, FAQs, dashboard, audit logs, settings.
EPIC 7 - Quality and Delivery	Tests, security review, documentation, deployment and presentation.

15. Non-Functional Requirements

Category	Requirement
Usability	Core workflows should be understandable without technical knowledge and should use consistent AdminLTE2 components.
Performance	Normal list and detail pages should respond promptly with pagination and indexed queries.
Reliability	Ticket creation must succeed even when AI analysis fails.
Maintainability	Code must be modular, documented, reviewed and covered by automated tests.
Scalability	The design should allow web and AI services to be scaled independently in future.

Category	Requirement
Accessibility	Forms require labels, keyboard usability, readable contrast and meaningful validation messages.
Observability	Log errors and important service events without exposing sensitive data.
Compatibility	Support current desktop browsers and responsive tablet/mobile layouts where AdminLTE2 permits.
Data Integrity	Use transactions for multi-step workflow changes and preserve history.
Backup/Recovery	Document MySQL backup and restore procedure for deployed environments.

16. Security and Privacy

- Use Django password hashing and built-in authentication controls.
- Enable CSRF protection for browser forms and secure cookie settings in production.
- Validate and sanitize all user input; rely on ORM parameterization rather than raw SQL.
- Enforce authorization at view, API and service layers.
- Prevent requesters from accessing internal notes or other users' tickets.
- Keep database passwords, API keys and Django secret key in environment variables.
- Limit login attempts or add throttling if feasible.
- Never log passwords, access tokens or full sensitive ticket content unnecessarily.
- Apply file type and size restrictions if attachments are implemented.
- Record privileged administrative changes in audit logs.
- Use HTTPS for deployed environments.

17. Testing Strategy

Test Level	Coverage
Unit Tests	Models, services, permission helpers, validators, transition rules, AI client and fallback behavior.
API Tests	Authentication, RBAC, valid/invalid requests, response schema, 401/403/404 behavior.
Integration Tests	Ticket creation through AI analysis, assignment, conversation, resolution and closure.
UI/Functional Tests	Login, create ticket, filters, agent reply, manager assignment and admin operations.
AI Evaluation	Hold-out examples, confusion matrix or accuracy, confidence review, retrieval relevance and failure cases.
Security Tests	Unauthorized access attempts, object-level leakage, CSRF, input validation and secret handling.
Deployment Smoke Tests	Health endpoints, login, database migration, sample ticket creation and AI call.

17.1 Minimum Critical Test Scenarios

40. Requester cannot view another requester's ticket.
41. Requester cannot see internal notes.

- 42. Agent cannot assign tickets without permission.
- 43. Manager can assign and reassign a ticket and history is recorded.
- 44. Ticket creation succeeds when FastAPI is unavailable.
- 45. Low-confidence AI category is visible as a recommendation.
- 46. Agent can edit an AI-suggested answer before sending.
- 47. Invalid status transition is rejected.
- 48. Resolved ticket can be reopened only according to policy.
- 49. Admin role change is written to the audit log.

18. Deployment and DevOps

- Provide .env.example without real secrets.
- Use Dockerfiles for Django and FastAPI and Docker Compose for local development.
- Run Django migrations as an explicit deployment step.
- Collect static files for production and configure the chosen web server/reverse proxy.
- Expose /health endpoints and verify service dependencies.
- Add GitLab CI stages for lint, test, build and optional deployment.
- Tag a release candidate before the final demonstration.
- Prepare sample users and seed data for the demonstration environment.
- Document backup, restore and rollback procedures.

19. Project Delivery Plan

Week	Primary Outcome
1	Onboarding, Git/GitLab/Jira, Docker fundamentals and approved project understanding.
2	Requirements, user stories, workflows, ERD, API contract, architecture and wireframes.
3	Django/FastAPI/MySQL foundation, AdminLTE2 conversion, authentication and ticket CRUD.
4	AI prototypes and prepared FAQ/ticket datasets.
5	FastAPI endpoints and integrated Django AI workflow with graceful fallback.
6	Complete assignment, conversation, status, dashboard and admin workflow.
7	Automated testing, security review, model evaluation, bug fixing and release candidate.
8	Deployment, documentation, rehearsal, final demonstration and evaluation.

19.1 Team Ownership

Area	Primary Owner	Team Expectation
Django application and integration	Software-development-focused intern	Others contribute UI, tests and reviews.
Classification and priority	AI intern	Expose tested FastAPI endpoints and document evaluation.
FAQ retrieval and suggested answers	AI intern	Prepare knowledge base and source-grounded responses.

Area	Primary Owner	Team Expectation
Summarization, sentiment and analytics	AI intern	Integrate outputs, dashboard metrics and tests.
Shared responsibilities	All interns	Git discipline, code reviews, documentation, demos and issue updates.

20. Acceptance Criteria and Definition of Done

20.1 Product Acceptance Criteria

- All roles can sign in and see only permitted navigation and data.
- A requester can create, view and reply to a ticket.
- A manager can triage, assign, categorize and prioritize tickets.
- An agent can communicate, add internal notes, use AI suggestions and resolve tickets.
- AI analysis is stored, displayed with confidence, and can be overridden by authorized staff.
- FAQ suggestions show their source and are not sent automatically.
- Ticket history and audit logs demonstrate the complete lifecycle.
- Admin users can manage users, categories and FAQ content.
- Dashboard data is generated from MySQL records.
- The complete project starts using documented Docker commands.
- Automated tests cover the critical workflows and RBAC rules.
- The deployed or local demonstration works end to end.

20.2 Definition of Done for a Jira Story

- Acceptance criteria are implemented and demonstrated.
- Code follows project structure and naming conventions.
- Permissions and validation are included.
- Automated tests are added or updated.
- No real secrets are committed.
- A merge request is created and reviewed by another team member.
- Review comments are resolved.
- Documentation is updated where required.
- The Jira issue contains evidence such as screenshots, test results or demo notes.

21. Future Enhancements

- Email and WhatsApp notifications.
- File attachments with malware scanning.
- Multi-tenant organizations and separate support teams.
- SLA policies and escalation timers.
- Real-time updates using WebSockets.
- Customer satisfaction surveys.
- Multilingual ticket translation and response suggestions.
- Advanced analytics and forecasting.
- Integration with external identity providers and enterprise help-desk tools.
- Human feedback loop for continuous model improvement.

22. Glossary

Term	Meaning
RBAC	Role-Based Access Control: permissions granted according to user roles.
Requester	A user who submits and tracks a support ticket.
Agent	A support team member who works on assigned tickets.
Triage	Initial review, categorization, prioritization and assignment.
FAQ	Frequently Asked Questions used as an approved knowledge source.
RAG	Retrieval-Augmented Generation: generating an answer using retrieved source content.
Confidence	A score representing the AI model's certainty; it is not a guarantee.
MVP	Minimum Viable Product: the required usable first release.
Audit Log	Immutable or controlled record of important actions and changes.
Docker Compose	A tool for running multiple related containers as one environment.

Project Approval and Working Agreement

The team should review this document during the requirements phase. Questions, assumptions and approved changes should be recorded in Jira. Development begins from the agreed MVP and proceeds through small, reviewable merge requests.

Role	Name / Signature	Date
Mentor / Product Owner		
Intern 1		
Intern 2		
Intern 3		
Intern 4		